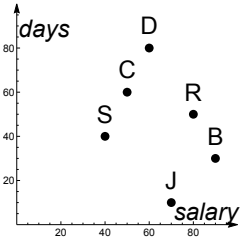
### A. Ranking (8 points)

The HR division of a company is screening job applicants based on their salary request (in K€) and availability to start a.s.a.p. (in days from today). There is just one available position, so they will retain only the best candidates.

1. How can HR discard unsuitable candidates? Discuss your idea and show the result and the steps to obtain it.
2. What is the proper notion to use in case there are two available positions? Indicate the suitable candidates.

| | sal. | days |
|---|---|---|
| Bonnie: | 90 | 30 |
| Connie: | 50 | 60 |
| Donnie: | 60 | 80 |
| Johnny: | 70 | 10 |
| Ronnie: | 80 | 50 |
| Sonny: | 40 | 40 |




**Solution.**

1. The safe way to proceed is to keep all the applicants that may be the best choice for some monotone scoring function using salary and availability (no specific scoring function is provided), i.e., the potentially optimal candidates. The skyline is exactly what we need in this case. We can compute it, e.g., via SFS, by first pre-sorting the dataset according to any monotonic scoring function of the available attributes. The table shows the candidates sorted by salary (there are no ties). Six dominance tests are sufficient to determine that Sonny and Johnny are the only two candidates in the skyline, as Connie, Donnie and Ronnie are dominated by Sonny, while Bonnie is dominated by Johnny.

| | sal. | days |
|---|---|---|
| Sonny: | 40 | 40 |
| Connie: | 50 | 60 |
| Donnie: | 60 | 80 |
| Johnny: | 70 | 10 |
| Ronnie: | 80 | 50 |
| Bonnie: | 90 | 30 |

2. If there are two positions, then we are looking for the potential top-2 candidates in the dataset, i.e., we should compute the 2-skyband, which consists of the set of candidates that are dominated by less than two candidates. Besides Sonny and Johnny, who are in the skyline, Connie (dominated only by Sonny) and Bonnie (dominated only by Johnny) also qualify; instead, Donnie (dominated by both Sonny and Connie) and Ronnie (dominated by both Sonny and Johnny) should be discarded.

### B. Concurrency Control (8 points)

Classify the following schedule for the classes: 2PL, strict 2PL, TS-mono, and TS-multi. Clearly motivate your answers. w1(x), r1(z), r2(x), r3(x), r1(y), w3(y), w3(z), r2(y), r1(x), w2(y)

**Solution.**

T1 can acquire all the resources at the beginning of the transaction. After the execution of $w_1(x)$, T1 can *downgrade* the lock on $x$ so that later the operation $r_1(x)$ can occur. This release is needed to allow T2 and T3 to execute their read operations on $x$. After $r_1(y)$, T1 must release also its lock on $y$ so that T3 can acquire the needed lock. Thanks to these two anticipated releases by T1, the schedule is 2PL, but cannot be strict 2PL (T1 must release its resources before the commit).

The schedule is not TS mono: according to this schedule on $y$ T2 should read a value written by T3, whereas it should be T2<T3. The other operations are already in the correct order according to the TS technique. Considering the TS multi version: on $y$ T2 can read the value written before the execution of $w_3$; $w_2$ cannot be executed after $w_3$ (unless Thomas' rule applies)

## Detailed analysis for the TS Mono class

| Operation | RTM($x$) | WTM($x$) | RTM($y$) | WTM($y$) | RTM($z$) | WTM($z$) | Killed transactions |
|---|---|---|---|---|---|---|---|
| $w_1(x)$ | 0 | 1 | 0 | 0 | 0 | 0 | |
| $r_1(z)$ | | | | | 1 | | |
| $r_2(x)$ | 2 | | | | | | |
| $r_3(x)$ | 3 | | | | | | |
| $r_1(y)$ | | | 1 | | | | |
| $w_3(y)$ | | | | 3 | | | |
| $w_3(z)$ | | | | | | 3 | |
| $r_2(y)$ | | | | | | | 2 as $2 <$ WTM($y$) $= 3$ |
| $r_1(x)$ | 3 | | | | | | |
| $w_2(y)$ | | | | | | | 2 (already killed) |

## Detailed analysis for the TS Multi class

| Operation | RTM($x$) | WTM($x$) | RTM($y$) | WTM($y$) | RTM($z$) | WTM($z$) | Killed transactions |
|---|---|---|---|---|---|---|---|
| $w_1(x)$ | 0 (0) | 0, 1 | 0 (0) | 0 | 0 (0) | 0 | |
| $r_1(z)$ | | | | | 1 (0) | | |
| $r_2(x)$ | 2 (1) | | | | | | |
| $r_3(x)$ | 3 (1) | | | | | | |
| $r_1(y)$ | | | 1 (0) | | | | |
| $w_3(y)$ | | | | 0, 3 | | | |
| $w_3(z)$ | | | | | | 0, 3 | |
| $r_2(y)$ | | | 2 (0) | | | | |
| $r_1(x)$ | 3 (0) | | | | | | |
| $w_2(y)$ | | | | | | | 2 as $2 <$ last WTM($y$) $= 3$ |

### C. Triggers (9 points + bonus)

A shop stores user preferences about the items it sells. Relation `pref(usr,btr,wrs)` indicates that user `usr` considers item `btr` better than item `wrs`. The recursive view `tPref` transitively closes `pref`:

```
CREATE RECURSIVE VIEW tPref (usr,btr,wrs) AS ( -- reported for completeness
  SELECT usr,btr,wrs FROM pref UNION           -- no need to analyze this code
  SELECT p.usr,t.btr,p.wrs FROM pref p JOIN tPref t ON t.wrs = p.btr AND t.usr = p.usr);
```

So, if `('Joe',1,2)` and `('Joe',2,3)` are in `pref`, then `tPref` also contains `('Joe',1,3)`. If `('Joe',3,1)` is also added to `pref`, then, among others, `tPref` contains `('Joe',1,1)`, which is an unwanted situation that indicates a "circular" preference. Design a system of triggers that incrementally reacts to `INSERT` and `DELETE` events in the `pref` table so as to avoid, for every user, the onset of circular preferences.
**Bonus.** Extend your triggers to also react to `UPDATE` events.

**Solution.** Each `btr-wrs` pair in `pref` can be regarded as an edge in a graph, and each user has an associated graph. If we keep all the graphs acyclic, then, clearly, no edge deletion can cause cycles, so no trigger is needed for `DELETE` events. Whenever a new edge `a-b` is inserted for a given user `u`, there are only two cases in which a new cycle is caused: (i) a path connecting `b` to `a` exists for user `u`, or (ii) `a` and `b` coincide.

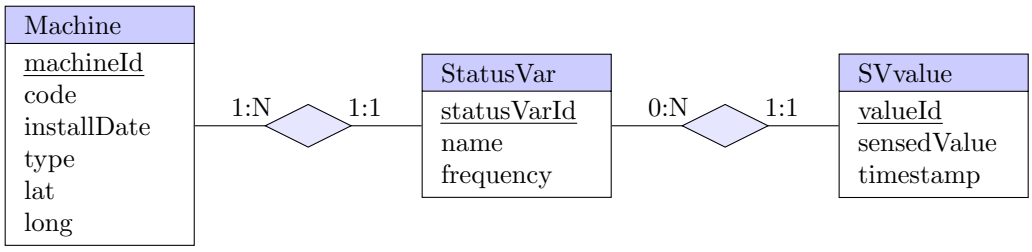
```
CREATE TRIGGER AcyclicPrefs AFTER INSERT ON pref
FOR EACH ROW
  IF new.wrs=new.btr OR EXISTS (
            SELECT * FROM tPref
            WHERE btr=new.wrs AND wrs=new.btr AND usr=new.usr)
  THEN
    RAISE EXCEPTION 'Circular prefs for user %', new.usr;
  END IF;
```

`UPDATE` events are more tricky, because the system will (very likely) not update `tPref` before the execution of the trigger, so things may go wrong. If, say, `('Joe',1,2)` is changed to `('Joe',2,1)`, then `tPref` will still contain `('Joe',1,2)`, which then would create a cycle with the new tuple. Therefore, we cannot rely on `tPref` and need to recompute on the fly (the relevant part of) the transitive closure of `pref`.

```
CREATE TRIGGER AcyclicPrefsUpdate AFTER UPDATE ON pref
FOR EACH ROW
  IF new.wrs=new.btr OR EXISTS (
        WITH RECURSIVE ontheflytPref AS ( -- recompute relevant part of tPref on the fly
        SELECT btr,wrs FROM pref WHERE usr=new.usr
        UNION
        SELECT t.btr,p.wrs FROM pref p
        JOIN ontheflytPref t ON t.wrs = p.btr AND new.usr = p.usr)
    SELECT * FROM ontheflytPref WHERE btr=new.wrs AND wrs=new.btr)
  THEN
    RAISE EXCEPTION 'Circular prefs for user %', new.usr;
  END IF;
```

### D. JPA (7 points)

An application manages the data collected via an IoT architecture from industrial machines for anomaly detection purposes. The user can inspect a list of machines and see the status variables that are collected by the IoT sensors associated with the machine. A machine has a code (int), a type (string), an installation date (date), and a geolocation (two float values for latitude and longitude). A status variable has a name (string) and a sampling frequency (e.g., one value per minute, hour, day) coded as an integer value. Each status variable is associated with multiple values read by an IoT sensor. A sensed value is a floating point number associated with a timestamp (DateTime).




The user can select one machine, then select one variable and open a histogram that shows the sensed values over time.

Machines are in the order of hundreds, variables per machine are in the order of tens, sensed values per variable are in the order of thousands.

Show the JPA entities (with all their attributes) that map the domain objects of the conceptual model, taking into account the above mentioned access paths of the application and data cardinalities. When designing the annotations for the relationships, specify the owner side of the relationship, the mapped-by attribute, the fetch policy and the cascading policies you consider more appropriate to support the access required by the web application. Comment on the design choices you have made.